Product Requirements Document (PRD)

LLM Observability Platform

Polaris School of Technology

On-the-Job Training (OJT) Project

B.Tech CSE (AI/ML) — Batch 2025–2029

Date: September 2026

Project Title	LLM Observability Platform
Team Members	Shuban Mutagi, Sagar Halladakeri
Mentor	Tanmay
Track	AI/ML — Generative AI
Version	1.0

Product Goal

Create an LLM observability platform that gives developers and AI teams real-time visibility into their LLM-powered applications — capturing traces, measuring output quality, tracking costs, and surfacing problems before users notice them.

Feature Priorities

Feature	Priority
SDK — auto-capture LLM calls (prompt, completion, tokens, latency, cost)	Must
Trace ingestion API — receive and store traces from the SDK	Must
Trace Explorer — search, filter, and view individual LLM call details	Must
Analytics dashboard — latency, error rate, token usage, cost over time	Must
Project management — isolated workspaces with API keys per project	Must
User authentication — register, login, role-based access	Must
Evaluation framework — score LLM outputs on relevance, faithfulness, safety	Should
Alerting — notify on latency, cost, or error rate threshold breaches	Should
Trace comparison — side-by-side view of two prompt versions	Should
Batch evaluation — re-run evaluators on historical traces	Could
Webhook notifications (Slack/Discord)	Could

User Stories

US-001

As a developer, I want to instrument my LLM calls with minimal code so that every call is automatically traced without disrupting my application.

Acceptance criteria

- SDK is installable via pip in one command.
- Wrapping an existing LLM call requires fewer than 5 lines of code change.
- Captured data includes: prompt, completion, model name, provider, token counts (prompt + completion), latency in ms, estimated cost, and error details.
- SDK does not block the main application thread.
- Traces are batched and flushed asynchronously to the backend.

US-002

As a team lead, I want to create a project so that my team's traces are isolated under a unique API key.

Acceptance criteria

- Project receives a unique ID and API key on creation.
- API key is required for all trace ingestion.
- Traces are scoped to the project — no cross-project data leakage.
- Project status starts as ACTIVE.

US-003

As an AI engineer, I want to search and filter my LLM traces so that I can find specific calls and debug issues quickly.

Acceptance criteria

- Traces are listed with model, timestamp, latency, token count, status, and cost.
- Filters available: model, time range, latency range, status (success/error), and custom tags.
- Clicking a trace opens the full detail view: prompt text, completion text, all metadata, and evaluation scores.

US-004

As an AI engineer, I want to see latency percentiles, error rates, token usage, and cost over time so that I can detect performance regressions.

Acceptance criteria

- Dashboard shows: p50, p95, p99 latency, error rate, request volume, total tokens, and total cost.
- Charts support time range selection (last 24h, 7d, 30d).
- Metrics can be broken down by model or custom tag.

US-005

As an AI engineer, I want my LLM outputs automatically scored on quality dimensions so that I can detect prompt regressions without manual review.

Acceptance criteria

- Built-in evaluators: relevance, faithfulness, toxicity/safety, format compliance.
- LLM-as-judge evaluation scores outputs against a configurable rubric.
- Evaluation scores are stored alongside the trace they evaluated.
- Evaluation results are visible in the trace detail view.

US-006

As a team lead, I want to configure alert rules so that I am notified when my LLM application exceeds cost, latency, or error thresholds.

Acceptance criteria

- Alert rules are configurable: metric, operator, threshold, and time window.
- Supported channels: email and webhook.
- Alert history is visible with trigger time, observed value, and resolution status.

US-007

As a user, I want to register and log in securely so that only my team can access our project's trace data.

Acceptance criteria

- Registration requires email and password.
- Login returns a session token (JWT).
- Roles supported: Admin, Member, Viewer.
- Unauthorized requests return 401 or 403.

Product States

`CAPTURED → INGESTED → STORED → EVALUATED → ALERTED (if threshold breached)`

Failure state: INGESTION_FAILED

Product Principles

- Instrument first, ask questions later — capturing data must be effortless. Teams won't observe what's hard to set up.
- Data, not opinions — the platform surfaces telemetry. What counts as "good" output is the team's call, not the platform's.
- Show your work — every evaluation score must trace back to the evaluator, the model version, and the input. No unexplained numbers.
- Cost is a first-class metric — token usage and spend are tracked at the same level as latency and errors.
- Degrade gracefully — if the backend is unreachable, the SDK buffers locally and retries. Traces should not be silently dropped.

End of PRD